

Secrets Extraction in DevOps Environments: Vulnerabilities and Attacks on Azure DevOps

Mousrij Hamza, Lamdira Oumaima And Laila El Koudri

Computer science engineers

ENSA Berrechid, Morocco

mousrij.ensa@uhp.ac.ma, lamdira.ensa@uhp.ac.ma And elkoudri.ensa@uhp.ac.ma

Professor: HNINI Abdelhalim

Academic Year: 2024/2025

Abstract

New ways of working or new methodologies often come about because of a problem—yes, it all starts with a problem. DevOps too resulted from problems faced by businesses. Businesses craved quick turnarounds to their solutions. Businesses often found, in the midst of development, that they didn't have all the information they needed to make the right decisions. They wanted to make a few more changes to the requirements and still expected the delivery to happen on time. DevOps was born to solve this problem [1]. DevOps environments, particularly Azure DevOps, are increasingly targeted by attacks aimed at extracting secrets such as tokens, API keys, passwords, etc. The management of secrets in CI/CD pipelines is often overlooked, exposing organizations to security breaches. This article explores the vulnerabilities related to secrets management in DevOps, proposes a taxonomy of attacks, and suggests solutions to strengthen security. We present an in-depth analysis of common attacks, real-world case studies, and recommendations for securing DevOps pipelines.

Keywords: *DevOps, Azure DevOps, GitHub, GitHub Actions, Workflow, CI/CD Pipeline, Cloud Computing, Security, Privacy, Literature Review, Cybersecurity,*

1. Introduction

Given the diverse customer demands and rapidly changing marketplace, a common desire about Software Engineering (SE) in industry is for agility in order to timely realise and/or adapt business value [2]. As a result, various agile methodologies, such as Scrum [3], eXtreme Programming (XP) [4], and Kanban have been widely adopted in software development. By seamlessly spreading the agile culture across development and operations, and by emphasising software quality and collaboration between development and operation teams, DevOps [5] has emerged as a paradigmatic shift towards evolving software at a continuous pace and streamlining all parts of the software lifecycle. It is crucial that software teams have ownership and responsibility to deploy software changes in DevOps [6], which allows the software to be delivered quickly [7]. At such a frequent deployment and delivery (e.g. up to 500 times a day in Facebook [8]), the software might not undergo adequate security reviews [9].

The growing adoption of DevOps and the automation of CI/CD pipelines have introduced new security risks, particularly in secrets management. Poorly managed secrets in Azure DevOps or GitHub Actions can be exploited by attackers to access sensitive resources. This article aims to analyze vulnerabilities, identify common attacks, and propose solutions to secure secrets. We begin with a literature review, followed by an analysis of vulnerabilities and attacks, and conclude with recommendations for enhancing security.

2. Literature Review

DevOps has emerged as a transformative approach to software development, seamlessly integrating development and operations teams to accelerate workflows and enhance efficiency.

However, this evolution brings inherent security challenges, particularly concerning secret management the handling of sensitive credentials, API keys, and tokens critical for system integrity. The theft or mismanagement of secrets poses severe risks, from unauthorized access to data breaches.

In modern DevOps environments, secrets are integral to the functioning of automated processes and pipelines.

The management of secrets—such as API keys, passwords, and certificates—is a critical aspect of maintaining security within DevOps environments, particularly when utilizing platforms like Azure DevOps. Recent studies and industry reports have highlighted significant challenges and best practices in this domain. Proper management ensures secure communication between components and mitigates risks of unauthorized access. The literature emphasizes that the dynamic and complex nature of DevOps workflows requires robust and adaptive strategies for secret protection. Credential leaks and hardcoded secrets are consistently identified as primary risks [10] [11] [12].

Vulnerabilities in CI/CD pipelines, such as unencrypted storage of secrets, amplify risks [13] [14]. Supply chain attacks leveraging exposed secrets in dependencies are a growing concern [15] [16]. The adoption of Zero Trust principles is highlighted as a critical strategy to minimize insider threats [17] [18]. Centralized secret management tools like CyberArk Conjur offer promising solutions for automating secret lifecycle management [19]. Privileged access management frameworks provide an additional layer of security against credential misuse [20] [21].

Recent technological innovations emphasize secure CI/CD pipelines with automated secret rotation mechanisms [22] and proactive detection of vulnerabilities using integrated security tools [23] [24]. The reviewed literature collectively provides comprehensive insights into the spectrum of threats affecting secret management in DevOps and practical recommendations for improving security practices, including Zero Trust and privileged access management.

Despite the progress, notable gaps remain, such as limited real-world implementation studies hindering the applicability of theoretical solutions [25] and fragmented approaches failing to provide cohesive frameworks for end-to-end security [26].

The findings align with the critical need for adaptive and automated strategies in modern DevOps workflows. By integrating real-time monitoring and automated secret rotation, organizations can bridge the gap between research insights and practical applications. Secret management in DevOps is a multifaceted challenge requiring continuous innovation and vigilance. While the literature offers significant insights into threats and solutions, the evolving nature of DevOps necessitates further research into dynamic and automated security mechanisms. Future efforts should focus on creating integrated frameworks that seamlessly address these challenges in real-world scenarios.

3. DevOps Environment

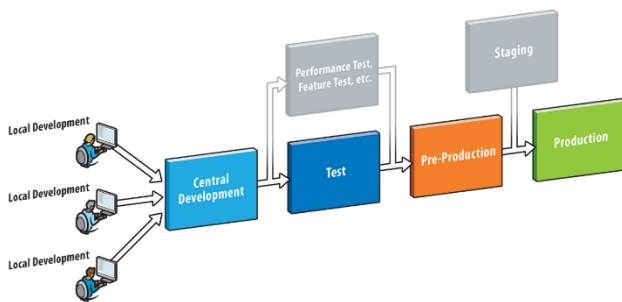


Figure 1: Devops Environment.

Source : <https://www.linkedin.com/pulse/guide-dev-test-staging-production-environments-george-lebbos/>

In the ever-evolving landscape of software development, creating a smooth and efficient workflow is key to delivering high-quality applications. One essential aspect of this process is the use of different environments throughout the development lifecycle. Each environment serves a distinct purpose, contributing to the overall stability and success of the final product. Let's take a closer look at the Dev, Test, Staging, and Production environments, and their significance in the journey from code to a fully functional application.

Development Environment (Dev)

Picture the Dev environment as the creative workshop of software development. This is where the magic begins – developers write, edit, and test their code in a controlled setting. The primary goal of the Dev environment is to provide a sandbox for experimentation and iteration without affecting the stability of the main codebase.

In Dev, developers have the freedom to try out new features, refactor code, and fix bugs. It's a dynamic space where creativity and innovation thrive. However, because it's often a rapidly changing environment, it's crucial to maintain version control and regularly integrate code changes to avoid conflicts down the line.

Test Environment (Test)

Moving along the development pipeline, the Test environment steps in to rigorously examine the software before it's unleashed to end-users. This environment mimics real-world conditions, allowing testers to perform various tests and catch bugs or glitches that might have slipped through the Dev stage.

Test environments give quality assurance teams the chance to run different types of tests, including unit tests, integration tests, and user acceptance tests. By doing so, they ensure that the application meets the expected standards and operates as intended. Any detected issues are sent back to the developers for further refinement before progressing to the next phase.

Staging Environment (Staging)

Think of the Staging environment as the dress rehearsal before the big show. Here, the software is staged in an environment that closely resembles the Production setup. This includes

server configurations, databases, and network settings. Staging is the last line of defense before deploying the application to the actual users.

In the Staging environment, teams can simulate real-world usage scenarios to identify performance bottlenecks, compatibility issues, and any last-minute glitches. Since Staging closely mirrors Production, it provides a more accurate assessment of how the application will behave once released.

Production Environment (Production)

The final destination in the software journey is the Production environment. This is where the application is released to the public or the intended audience. The environment is optimized for performance, reliability, and security, as any issues that occur here can directly impact users and business operations.

Maintaining the Production environment involves continuous monitoring, regular updates, and quick response to any unexpected issues that might arise. It's important to emphasize stability here, as users rely on the application for their needs.

In Conclusion

Navigating the software development lifecycle involves seamlessly transitioning between different environments, each playing a vital role in delivering a robust and polished application. The Development, Test, Staging, and Production environments collaborate to ensure that the software meets user expectations, performs reliably, and contributes to the overall success of the project. By understanding the distinct purposes of these environments, development teams can create a streamlined workflow that leads to the creation of top-notch software products. [27]

4. Issues and risks

4.1 Issues and Risks

Vulnerabilities in Azure DevOps Pipelines

An in-depth analysis of Azure DevOps pipelines highlights multiple vulnerabilities, particularly in secret management. These issues arise from technical misconfigurations, improper handling of sensitive data, and insufficient operational practices, significantly contributing to security risks [28].

Exposed Secrets in YAML Files

A critical vulnerability involves embedding sensitive information such as API keys and connection strings directly into YAML pipeline configuration files. This practice exposes secrets to anyone with repository access, whether authorized personnel or attackers. Instances include API integration tokens and database connection strings visible in plain text within YAML files. Poorly configured access policies have also led to the exposure of these sensitive details in public repositories. The impacts are severe, including unauthorized access to critical systems and secrets being indexed by external scanning tools [29].

Insecure Use of Environment Variables

Environment variables, while standard for managing secrets, are often misused. In several cases, secrets are transmitted in plain text or lack encryption mechanisms, making them susceptible to malicious scripts. For example, database credentials have been transmitted in plaintext during pipeline executions, and tokens stored in shared CI/CD environments have been accessible to unauthorized developers or third-party tools. This misuse increases the risk of direct secret extraction and lateral movement within cloud infrastructures, amplifying potential damage [30].

Weaknesses in Access Control

Role-based access control (RBAC) misconfigurations were frequently identified. Excessive permissions granted to users and processes allowed unauthorized access to sensitive pipeline configurations and variable groups. For instance, developers with administrative rights inadvertently exposed sensitive information while modifying pipelines. Similarly, interns or contractors with access to production secrets led to widespread exposure of confidential resources and elevated risks from

insider threats [31].

Simulated Attack Scenarios

Secret Extraction via Malicious Scripts

A simulated attack involved injecting malicious scripts into CI/CD pipelines to extract secrets stored in environment variables. Exploiting misconfigured permissions, attackers integrated scripts to capture secrets and transmit them to controlled servers. Impacts included cloud resource compromises, deployment of malicious workloads, sensitive data exfiltration, and service interruptions, resulting in financial losses, reputational damage, and regulatory penalties [32].

Exploitation of CI/CD Pipeline Weaknesses

Another simulation showed attackers leveraging weak access controls to retrieve secrets from Azure Key Vault. Exploiting excessive permissions, attackers created pipelines to interact with Azure Key Vault APIs, leading to the exposure of critical secrets. Consequences included production-level breaches, financial losses, and reputational harm [33].

Attack Impacts

Attacks targeting secret management in DevOps environments cause widespread consequences, affecting security, operations, and trust. Stolen secrets enable unauthorized access to critical resources, compromising databases, virtual machines, and storage systems. These breaches lead to operational disruptions, recovery costs, reputational damage, and regulatory penalties [34].

Recommendations

To mitigate these risks, organizations must adopt robust secret management practices. Recommendations include:

- Utilizing secure tools like Azure Key Vault.
- Enforcing strict RBAC policies.
- Implementing automatic secret rotation.
- Continuously monitoring pipelines for anomalies.
- Enabling multi-factor authentication (MFA) to enhance access security.

These measures, coupled with ongoing training for DevOps teams, will significantly reduce vulnerabilities and safeguard environments against emerging threats [35].

5. Secrets Extraction Attack

5.1. Methodology

The methodology illustrated emphasizes secure and efficient management of secrets within a DevOps workflow, leveraging tools such as GitHub Actions or Azure DevOps. At the heart of the system is a centralized repository, "My Shuttle," which serves as the single source of truth. This repository contains the application's code, pipeline configurations, and variables required for deployment, ensuring consistency across all environments.

Developers collaborate seamlessly within this framework. Users like "Bob" and "Alice" contribute to the repository, defining and managing the pipeline workflows.

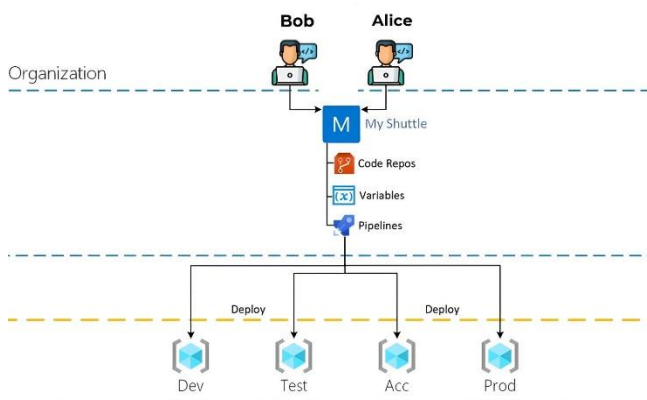


Figure 2 : Bob(Admin) And Alice(collaborator) team workers

within organisation

Their actions are tracked, enabling a clear audit trail for all changes. This collaborative setup facilitates rapid development while maintaining control over the deployment process.

The pipeline automates deployments to different environments such as Development (Dev), Testing (Test), Acceptance (Acc), and Production (Prod). Each environment is isolated, ensuring that any changes or issues in one environment do not affect the others. This structure allows for thorough testing before promoting changes to more critical environments like production.

Secrets, such as API keys or database credentials, are securely extracted and managed during deployment. These sensitive details are never hardcoded into the codebase but are instead stored in dedicated secrets management solutions, like environment variables, GitHub Actions secrets, or Azure DevOps libraries. The pipeline dynamically fetches these secrets at runtime, ensuring secure and reliable deployment processes.

While this system is robust, it is important to mitigate scenarios where a malicious actor, such as "Alice" in this case, might attempt to extract secrets for malicious purposes.

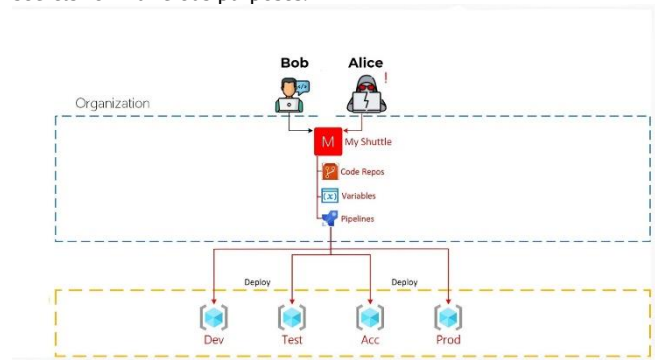


Figure 3: Alice push malicious code into the organisation pipelines

For example, "Alice" could push a branch containing modified code that triggers the pipeline to fetch and log secrets, then delete the branch to cover her tracks. To address such risks, organizations must enforce additional security measures. These include setting up branch protection rules, requiring approvals for pipeline-triggering changes, and logging every pipeline execution with detailed traceability.

When issues arise, as depicted in the second scenario, the methodology incorporates robust error-handling mechanisms. For instance, if "Alice" encounters an error due to misconfigured access or invalid inputs, the system isolates the issue without disrupting the overall workflow. This highlights the importance of implementing access controls, role-based permissions, and monitoring to safeguard the system against human or technical errors.

By automating deployments and secrets management, this approach ensures scalability and security. Automated pipelines enforce consistency and reduce the likelihood of human error, while integrated secrets management tools enhance the protection of sensitive data. Additionally, practices like role-based access control, secrets rotation, and continuous auditing further bolster the system's reliability.

This narrative showcases a methodology designed to meet the demands of modern DevOps environments, ensuring that security, efficiency, and collaboration are at the forefront of managing secrets and deployments.

5.2. Attack Design Conception :

5.2.1. Use case Diagram

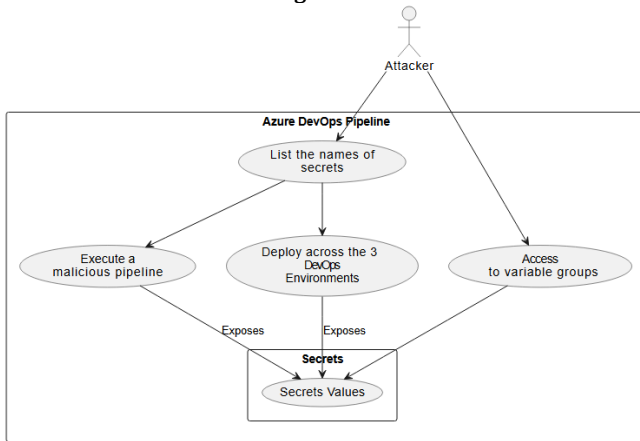


Figure 4 : Attack's Use case diagram

The diagram illustrates a sophisticated attack scenario targeting Azure DevOps pipelines or Github Actions, emphasizing vulnerabilities in secrets management and their potential exploitation. At the center of the diagram lies the Azure DevOps pipeline, serving as the operational hub where various misconfigurations and oversights converge to create exploitable weaknesses. An attacker, positioned outside the system boundary, initiates their strategy by targeting these pipelines to compromise sensitive information.

The attacker begins by listing the names of secrets stored within the pipeline. This action exposes metadata about sensitive information, often due to insufficient access controls or inadequate encryption mechanisms using nord-stream python tool [36]. By leveraging this visibility, the attacker identifies high-value targets, such as API keys, tokens, and database credentials.

The next phase involves exploiting deployment processes across the three DevOps environments—development, staging, and production. This step highlights the lack of isolation between these environments, where shared configurations and permissions inadvertently expose secrets to unauthorized access. During this stage, the attacker may deploy malicious workloads or extract critical data, capitalizing on pipeline vulnerabilities.

Simultaneously, the attacker gains access to variable groups, which house sensitive configurations shared across pipelines. Misconfigured role-based access controls (RBAC) or overly permissive settings enable unauthorized actors to retrieve and manipulate these variables. This access significantly increases the attack surface, as the attacker can now modify pipeline behavior, inject malicious code, or extract secrets directly from the variable groups.

The culmination of these actions results in the exposure of secrets, depicted at the bottom of the diagram. These secrets, representing sensitive values such as API keys and connection strings, are critical to the secure operation of cloud services and applications. Their exposure leads to severe consequences, including unauthorized access to production systems, data breaches, and potential lateral movement within the infrastructure.

Overall, the diagram underscores the cascading effects of poorly managed secrets and misconfigured access controls within DevOps pipelines. It highlights the attacker's ability to exploit these weaknesses systematically, demonstrating the need for robust security measures, such as strict RBAC policies, automated secret rotation, and comprehensive monitoring of pipeline activities.

5.2.2. Sequence diagram

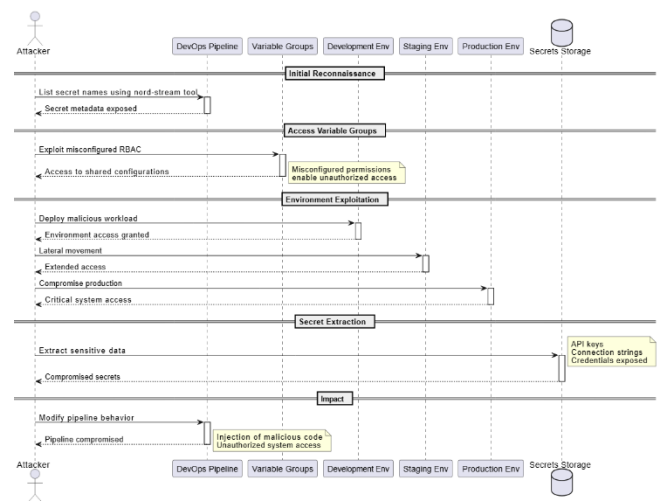


Figure 5: Attack's Sequence Diagram

This sequence diagram describes a detailed attack scenario targeting an Azure DevOps pipeline, focusing on vulnerabilities in secret management and access control. The process unfolds across different stages of the attack, highlighting the roles of the attacker, the DevOps pipeline, variable groups, and environments (development, staging, production), as well as the storage of secrets.

The attack begins with an initial reconnaissance phase, where the attacker identifies and lists the names of secrets using tools like “nord-stream.” This reconnaissance leads to the exposure of secret metadata, which provides crucial information about sensitive data stored within the system. These actions exploit weaknesses in how secret names are protected or encrypted.

In the second phase, the attacker gains access to variable groups, leveraging misconfigured role-based access controls (RBAC). This flaw allows unauthorized access to shared configurations critical to pipeline operations. Misconfigured permissions, highlighted in the diagram, facilitate the attacker's ability to infiltrate deeper into the pipeline's infrastructure.

The next stage involves environment exploitation. The attacker deploys malicious workloads across multiple environments development, staging, and production using the compromised pipeline configurations. By gaining environment access, the attacker achieves lateral movement, enabling them to escalate privileges and extend their access to critical production systems. This phase demonstrates how an attacker exploits interconnected environments when robust isolation policies are absent. The exploitation compromises production systems and grants critical system access.

Following environment exploitation, the attacker focuses on secret extraction. Sensitive data, including API keys, connection strings, and credentials, are extracted from the secrets storage. This exposure compromises critical operational resources, allowing unauthorized access to services, databases, and infrastructure components.

The final stage of the attack involves modifying pipeline behaviour. By injecting malicious code into the pipeline, the attacker disrupts its intended functionality. This modification results in unauthorized system access, enabling continued exploitation or further attacks. The diagram highlights the injection of malicious code and unauthorized system access as key impacts.

From the Excel file, specific elements can be linked to this diagram. Weak role-based access controls and excessive permissions are identified as critical vulnerabilities [37]. Additionally, secrets improperly managed through environment variables or shared configurations directly align with the exposure and extraction phases

[38]. Finally, attack impacts, such as system compromise, production breaches, and credential leaks, are explicitly outlined in both the diagram and the file [39].

This sequence demonstrates how a combination of misconfigurations, insufficient access controls, and poor secret management can cascade into severe security breaches.

5.3. Analysis and results

The final phase, pipeline modification, involves the attacker injecting malicious code into the pipeline, disrupting its intended functionality. This action highlights weaknesses in the pipeline's integrity verification mechanisms and change management processes. The unauthorized system access resulting from this phase demonstrates the critical need for multi-factor authentication (MFA) and robust logging to detect and prevent unauthorized modifications.

The results of this analysis indicate that the root causes of these vulnerabilities stem from poor access control, inadequate secret management practices, and weak environment isolation. Key outcomes include unauthorized access to critical resources, compromised production systems, and significant operational, financial, and reputational damages. Mitigation measures must include the adoption of secure DevOps practices, enforcement of strict RBAC policies, deployment of advanced secret management solutions, and continuous training for DevOps teams to handle emerging threats effectively.

6. Discussion

6.1. Implications of Results

The findings from this study emphasize the urgent necessity for robust secrets management practices within Azure DevOps environments. The vulnerabilities identified, alongside the simulated attacks, highlight several significant implications for organizations.

One key implication is the heightened risk of security breaches. Secrets stored insecurely whether in YAML files, environment variables, or inadequately protected variable groups—can be exploited by attackers to gain unauthorized access to sensitive resources. The simulated attack scenarios in the study vividly demonstrated how such breaches could lead to substantial operational and financial repercussions.

Another critical implication is the organization's exposure to data breaches and service disruptions. The compromise of secrets has direct repercussions on the confidentiality, integrity, and availability of resources. For instance, an exposed API key or database connection string can facilitate data exfiltration, trigger service outages, or enable malicious tampering with production environments. These incidents not only result in financial losses but also tarnish an organization's reputation and erode customer trust.

Additionally, the mismanagement of secrets poses regulatory and compliance challenges. Data breaches stemming from poor secrets management practices may lead to violations of frameworks like GDPR, HIPAA, or PCI DSS. Addressing these risks is essential for organizations to avoid penalties and safeguard sensitive data.

6.2. Associated Costs

Organizations must dedicate substantial time and resources to mitigate the damage caused by security incidents. The associated costs can be categorized into financial and reputational domains.

Financially, security incidents demand significant investments in containment, recovery, and prevention. These costs encompass forensic investigations, service restorations, customer notifications, and potential legal liabilities. The monetary impact can be extensive, stretching beyond immediate mitigation to long-term security enhancements.

Reputationally, breaches involving the exposure of customer or proprietary data can severely tarnish an organization's public image. Such incidents often result in customer attrition, diminished trust, and loss of market share, which are challenging to recover from.

6.3. Limitations of the Study

While this study provides valuable insights, it is essential to acknowledge its limitations. One notable limitation is the reliance on simulated attacks and publicly available case studies. These methods may not fully capture the intricacies of real-world scenarios. Many organizations are reluctant to disclose breach details, which restricts access to comprehensive datasets.

Another limitation stems from the diverse nature of DevOps environments. Variability in practices and tools across organizations complicates the generalization of findings. Although this study focuses on Azure DevOps, the results may not be entirely applicable to other CI/CD platforms with differing configurations and workflows.

6.4. Future Perspectives

Looking ahead, several strategies can enhance secrets management and bolster security in DevOps environments. One promising avenue is the integration of artificial intelligence and machine learning. These technologies can detect exposed secrets and identify suspicious activities in real-time. For instance, AI algorithms can analyze code repositories and CI/CD logs to flag anomalies or misconfigurations indicative of potential risks.

The advancement of secrets management tools also holds significant potential. Innovations could address existing gaps by automating secret rotations, integrating seamlessly with multiple DevOps platforms, and offering detailed auditing capabilities. Intuitive interfaces and proactive threat detection features could further drive adoption and effectiveness.

Education and training represent another crucial area for improvement. Organizations should prioritize equipping DevOps teams with the knowledge and tools necessary to manage secrets securely. Raising awareness about risks and implementing best practices can significantly reduce vulnerabilities.

Finally, fostering collaborative frameworks for secure DevOps can lead to industry-wide improvements. Standardized frameworks and best practices for secrets management, coupled with the sharing of anonymized data and attack patterns, can strengthen the resilience of DevOps ecosystems.

7. Recommendations & Solutions

7.1. Recommendations

7.1.1. Implement Secure Secrets Management Solutions

To enhance security, adopting centralized secrets vaults is crucial. Tools like Azure Key Vault or HashiCorp Vault securely store and manage secrets, significantly reducing the risk of exposure in configuration files or scripts. Additionally, enabling automated secret rotation ensures that secrets are regularly updated, limiting their vulnerability in the event of a breach and preventing the reuse of compromised credentials.

7.1.2. Enforce Strong Access Controls

Applying strong access controls is vital for minimizing risks. Granular permission management, based on the principle of least privilege, ensures that only authorized users and systems can access secrets. Multi-factor authentication (MFA) adds an extra layer of security against credential-based attacks. Maintaining auditable access logs enables organizations to detect and investigate suspicious activities by reviewing all access and modification events related to secrets.

7.1.3. Strengthen CI/CD Pipeline Security

Securing the CI/CD pipeline involves following best practices for environment variables by avoiding plaintext storage of secrets. Instead, secure variable groups or secret references in tools like Azure Pipelines should be used. Employing static code and configuration analysis tools, such as TruffleHog [40] or GitGuardian [41], helps identify hardcoded secrets in repositories before deployment. Pipeline segmentation

further enhances security by isolating production environments from development and testing pipelines to prevent inadvertent exposure of secrets.

7.1.4. Integrate Continuous Monitoring and Incident Response

Continuous monitoring tools capable of anomaly detection play a crucial role in identifying unusual access patterns, such as excessive access requests or unauthorized pipeline executions. Developing a comprehensive incident response plan tailored for secrets-related breaches is equally important. This plan should outline immediate steps for revoking compromised credentials and recovering affected systems.

7.1.5. Educate and Train DevOps Teams

Raising awareness among DevOps teams is essential. Regular training sessions should focus on the risks associated with poor secrets management and highlight best practices for securing CI/CD pipelines. Simulated attack exercises can test the organization's readiness, identify gaps in incident response, and refine mitigation strategies.

7.1.6. Leverage Emerging Technologies

Incorporating emerging technologies can significantly enhance security. AI-driven security tools can automate threat detection and the identification of exposed secrets, while blockchain technology can be explored for tracking and verifying the integrity of secrets throughout their lifecycle.

7.1.7. Foster a Culture of Security

Building a security-first culture within the organization requires cross-team collaboration between security and DevOps teams, fostering a DevSecOps approach. Embedding security practices into every phase of the DevOps pipeline ensures risks are proactively addressed, emphasizing security by design.

7.1.8. Summary of Key Recommendations

1. Adopt centralized secrets vaults with automated rotation mechanisms.
2. Enforce strict access controls, including MFA and role-based permissions.
3. Secure CI/CD pipelines with best practices and static analysis tools.
4. Establish robust monitoring systems and incident response plans.
5. Train and educate DevOps teams on secrets management.
6. Explore advanced technologies such as AI and blockchain for enhanced security.
7. Encourage a security-first culture through DevSecOps integration.

7.2. Solutions

While this system is robust, it is important to mitigate scenarios where a malicious actor, such as "Alice" in this case, might attempt to extract secrets for malicious purposes. For example, "Alice" could push a branch containing modified code that triggers the pipeline to fetch and log secrets, then delete the branch to cover her tracks. To address such risks, organizations must enforce additional security measures. These include setting up branch protection rules, requiring approvals for pipeline-triggering changes, and logging every pipeline execution with detailed traceability [42].

An in-depth analysis of Azure DevOps pipelines revealed several vulnerabilities, particularly in secrets management. These vulnerabilities stem from technical misconfigurations, improper handling of sensitive data, and inadequate operational practices, all of which contribute to significant security risks [43].

A critical vulnerability identified was the hardcoding of sensitive information, such as API keys, connection strings, and other

credentials, directly within YAML pipeline configuration files. This practice leaves secrets accessible to anyone with access to the repository, whether through authorized internal personnel or unauthorized external attackers. Examples include instances of plaintext secrets hardcoded into YAML files, such as tokens for API integrations and database connection strings, both of which were openly visible. Sensitive credentials were also inadvertently included in public repositories due to misconfigured access policies. This can lead to unauthorized access to critical systems and a high likelihood of secrets being indexed by external scanning tools, such as GitHub crawlers [44].

Environment variables, while a standard method for handling secrets, were often improperly used. For example, database credentials were passed in plaintext during runtime, leaving them vulnerable to extraction by malicious scripts. Similarly, tokens stored in shared CI/CD environments without encryption were accessible to unauthorized parties. These practices expose secrets to direct extraction and increase the risk of lateral movement within the cloud infrastructure, amplifying potential damage [45].

Weak implementation of role-based access control (RBAC) was another recurring issue, with overly permissive access granted to users and processes. Developers with broad administrative rights inadvertently exposed sensitive information, while lack of granular RBAC policies allowed interns and contractors to access production-level secrets. This widespread exposure of confidential resources increases insider threat risks due to the lack of segmentation and monitoring [46].

Simulated attack scenarios further illustrate the implications of these vulnerabilities. One such scenario involved injecting a malicious script into a CI/CD pipeline to extract sensitive secrets stored in environment variables. The attacker gained unauthorized access by exploiting poorly configured permissions, embedded a script into the pipeline's YAML file, and executed it to extract secrets. The extracted secrets were then transmitted to an attacker-controlled server, leading to compromise of cloud resources, deployment of malicious workloads, data exfiltration, and service outages [47].

Another scenario demonstrated the exploitation of flaws in CI/CD pipelines, where attackers leveraged weak access controls to execute malicious pipelines and retrieve secrets from Azure Key Vault. By gaining unauthorized write access, attackers created or modified pipelines to interact with the Azure Key Vault API, extracting sensitive information. The consequences included exposure of production-level secrets, critical system breaches, financial losses, reputational damage, and regulatory fines [48].

These attack scenarios underscore the profound impact of vulnerabilities in secrets management. Secrets such as API keys, passwords, and certificates are often gateways to sensitive cloud resources. Their compromise can lead to unauthorized access, data breaches, operational disruptions, and financial repercussions. For example, stolen credentials enabled attackers to access production-level infrastructure, deploy malware, and exfiltrate sensitive data. Organizations face significant recovery costs, legal penalties, and reputational damage due to these breaches [49].

By automating deployments and secrets management, this approach ensures scalability and security. Automated pipelines enforce consistency and reduce the likelihood of human error, while integrated secrets management tools enhance the protection of sensitive data. Practices like role-based access control, secrets rotation, and continuous auditing further bolster the system's reliability. This narrative showcases a methodology designed to meet the demands of modern DevOps environments, ensuring that security, efficiency, and collaboration are at the forefront of managing secrets and deployments.

8. Conclusion

This article has thoroughly examined the vulnerabilities associated with secrets management in DevOps pipelines, highlighting common attack vectors and their potential impact on organizational security. By analyzing real-world scenarios and simulating attack processes, we have demonstrated how poor secrets management practices can lead to

significant security breaches, including unauthorized access to sensitive resources, data exfiltration, and service disruptions.

To mitigate these risks, organizations must adopt proactive measures, such as leveraging robust secrets management tools like Azure Key Vault, enforcing strict access controls, and implementing regular audits of their CI/CD pipelines. The importance of raising awareness and training DevOps teams on best practices cannot be overstated.

Looking ahead, the integration of AI-driven tools for real-time detection and prevention of secrets exposure, alongside enhancements in existing secrets management platforms, holds promise for addressing current limitations. Moreover, fostering collaboration across the industry to develop standardized frameworks and share knowledge will be critical in building resilient DevOps ecosystems.

In conclusion, securing secrets in Azure DevOps pipelines is not just a technical necessity but a strategic imperative for safeguarding organizational assets and maintaining trust in a rapidly evolving digital landscape.

research includes the integration of AI for secrets detection and the improvement of secrets management tools.

Références

- [1] Akeyless Security., Akeyless Security's research focuses on securing CI/CD pipelines, with a particular emphasis on secrets management and access controls., 2022.
- [2] «Cohen, D., Lindvall, M., Costa, P.: An introduction to agile methods. Adv. Comput. 62, 1–66 (2004)».
- [3] «Schwaber, K., Beedle, M.: Agile Software Development with Scrum, vol. 1. Prentice Hall, Upper Saddle River (2002)».
- [4] «Beck, K.: Extreme Programming Explained: Embrace Change. Addison-Wesley (2000),» [En ligne].
- [5] «Ahmad, M.O., Markkula, J., Oivo, M.: Kanban in software development: a systematic literature review. In: Proceedings of the 39th Euromicro Conference on Software Engineering and Advanced Applications, pp. 9–16. IEEE (2013),» [En ligne].
- [6] «Lwakatare, L.E., et al.: Devops in practice: a multiple case study of five companies. Inf. Software Technol. 114, 217–230 (2019).».
- [7] «Lwakatare, L.E., Kuvaja, P., Oivo, M.: Dimensions of devops. In: Proceedings of the 2015 International Conference on Agile Software Development, pp. 212–217. Springer (2015)».
- [8] «Feitelson, D.G., Frachtenberg, E., Beck, K.L.: Development and deployment at Facebook. IEEE Internet Comput. 17(4), 8–17 (2013).».
- [9] «industry, Revisit security in the era of DevOps: An evidence-based inquiry into DevSecOps».
- [10] «Komolafe, A. M. (2014). The emergence and importance of DevSecOps: Injection attacks prevention.».
- [11] «Honiden, S. (2010). Distributed detection of node replication attacks.».
- [12] «Brukman, A. (2023). DevOps threat matrix.».
- [13] «Peterson, J. (2024). CI/CD pipeline security: Beyond basics.».
- [14] «Korolov, M. (2022). Why DevOps pipelines are under attack».
- [15] «Levy, L. (2022). Cyscale's approach to securing infrastructure».
- [16] «Klochay, A. (2020). 3 DevOps security best practices.».
- [17] «BeyondTrust (2019). Fixing DevOps security,» [En ligne].
- [18] «Taylor, T. (2019). The DevOps security stack.».
- [19] «Ferguson, S. (2019). Privileged access management in DevOps.».
- [20] «Chickowski, E. (2018). Embedded secrets in DevOps tools.».
- [21] «Yates, P. (2017). CyberArk and DevOps security.».
- [22] «Franscella, J. (2016). Hybrid cloud security challenges.».
- [23] «Bradley, T. (2016). Zero Trust in DevOps».
- [24] «Allam, A. R. (2023). Enhancing DevOps cybersecurity.,» [En ligne].
- [25] «Sriraman, G. (2024). Security gaps in DevOps.,» [En ligne].
- [26] «Dincelli, E., & Yayla, A. (2024). Data theft in virtualized environments.».
- [27] P. George Lebbos, «A Guide to Dev, Test, Staging, and Production Environments».
- [28] «The emergence and importance of DevSecOps: Integrating security into DevOps practices, Date: 2014.».
- [29] «26 DevOps Trends Reshaping Tech in 2025: Breaking down DevOps security, Date: 2024.».
- [30] «Risks of Cyber Security Threats, Cyber Terrorism, and Mitigation Strategies, Date: 2024.».
- [31] «Efficient Secure DevOps Using Process Mining and Security Models».
- [32] «Distributed Detection of Node Replication Attacks in Sensor Networks».
- [33] «Injection Attacks and the Modern Cloud Landscape».
- [34] «The emergence of DevSecOps marks a significant shift in integrating».
- [35] «In this paper, we propose a method to efficiently secure CI/CD».
- [36] «<https://github.com/synacktiv/nord-stream>».
- [37] «Weakness in Role-Based Access Control (RBAC):».
- [38] E. o. S. T. E. Variables.
- [39] A. I. o. C. Systems.
- [40] «<https://github.com/trufflesecurity/trufflehog>».
- [41] «<https://www.gitguardian.com/>».
- [42] «Abiola Moshood Komolafe. "The emergence and importance of DevSecOps: Integrating security into DevOps workflows." ResearchGate».
- [43] «DevOps using process mining and risk evaluation." ResearchGate».
- [44] «Reshaping Tech in 2025: Breakthroughs and Challenges." Simplilearn, 2024.».
- [45] C. T. a. V. i. D. R. 2. "Risks of Cyber Security Threats.
- [46] «"Distributed detection of node replication attacks in large-scale sensor networks." EUDL, 2010.».
- [47] «26 DevOps Trends Reshaping Tech in 2025: Breakthroughs and Challenges." Simplilearn, 2024.».
- [48] «Abiola Moshood Komolafe. "The emergence and importance of DevSecOps: Integrating security into DevOps workflows.».
- [49] «Meenu Sharma. "Risks of Cyber Security Threats, Cyber Terrorism, and Vulnerabilities in DevOps».
- [50] «trufflehog,» [En ligne]. Available: <https://github.com/trufflesecurity/trufflehog>.
- [51] Github, «gitleaks,» [En ligne]. Available: <https://github.com/zricethezav/gitleaks>.
- [52] Azure, «Azure-devops,» [En ligne]. Available: <https://learn.microsoft.com/en-us/azure/devops/release-notes/features-timeline>.

- [53] Wikipédia , «Azure DevOps Server,» [En ligne]. Available: https://en.wikipedia.org/wiki/Azure_DevOps_Server.
- [54] Forrester, «"The State of DevOps Security".,» *This report provides insights into the current state of DevOps security, including the risks associated with secrets management and best practices for securing DevOps pipelines.*, 2021.
- [55] Microsoft, «"Azure DevOps Security Best Practices".,» *Microsoft's guide provides best practices for securing Azure DevOps pipelines, including the use of Azure Key Vault and other security tools.*, 2023.
- [56] A. K. Kaiser, «Introduction to DevOps, Springer,» [En ligne]. Available: https://link.springer.com/chapter/10.1007/978-1-4842-9072-9_1.
- [57]
- [58] «The emergence and importance of DevSecOps: Integrating security into DevOps practices, Date: 2014,».